14. Anatomy of a Spring Boot Project

In the next lesson, we are going to be creating a Spring Boot project. But first, let's take a look at the anatomy of a Spring Boot project. Now, by default, Spring Boot is going to follow the Maven standardized directory layout. This was actually established by Maven with the 2.0 release back in 2005. It effectively became an industry standard for Java projects. Prior to that, it was kind of a free for all. Everything was a little bit different. They didn't have the standards and they were developing at the time, but this has really become more of a standard for Java projects. It is also used by other Java build tools such as Gradle. In the project directory, the root of your project, you will have a pom.xml file. This is the main Maven build file with project metadata dependencies, build plugins, and also a variety of things that you can do with Maven. You also typically will have a Readme. This is a markdown file that you can put project information in and and if you put it into GitHub, this will be shown on the root page of the GitHub repository project directory. Within Maven you will have a target directory. And this is where Maven is going to place all the build artifacts. So when it goes through and builds your application, all the build artifacts are going to be placed under target. The slash src. This is where all the source files are going to be under source main. This will be the application source files, particularly source main Java. This will be the Java source files and then source main resources. These are non Java source files such as property files or images. Also in the project directory you will have a source test. This is where you'll have the application files for test the source test Java. This will be the Java source files for unit test. And you may need resources. So you have source test resources. This will be non Java source files such as property files or images specifically for unit tests to help you visualize the project structure. And we'll have the main directory under the main directory will be a pom.xml. That's your build file, a Readme. And when we create a project with the spring boot initializer, we will have a. Gitignore file to ignore specific files and directories from GitHub. There are two directories under the project netlib source or src and also target. Within the source directory you'll have main and test, and then both in main and test you'll have Java and resources within those directories. Now Spring Boot does add in some specific files under source main resources. There will be either an

application.properties file or an application YAML. You really want to be using one or the other, not both, but in this case you have two different formats to specify properties to be used within the spring application. Also, within source test resources, you have the same application.properties or application YAML to provide properties files for unit tests. Then spring does have a couple special directories. You have source main resources. Static. This is where you can play static web files such as images, JavaScript or whatever that you need for a web application. And by default, Spring Boot is configuring the application to serve these files from the root URL. Then in source main resources templates. This is where you put web application template files. And then finally we have source main resources db migration. This is where you store database migration scripts. Now the default package of your project. Let's say you have a project called Myproject example. That is a Java convention. To use the reverse domain name of your organization, there will be a spring boot main class. The main class is annotated with the annotation spring boot application. This indicates to the framework. This is the main class that's going to be executed at startup time. The main method is executed to start the spring boot application, and this is where it kicks off all the auto configuration and starts up the application. It is very important that Spring boot is going to scan for components Bonus within the default package and all child packages. I see a common mistake where people add spring components outside of the default package, and these are not picked up by default. You can add additional configurations to pick up components outside of the default package, but by default it is not. Projects created using the spring initializer will have a default test for the application. This is a very simple test just to verify that spring is able to start without errors. Here is an example of a spring boot application. This is where you have the main class and you can see it is annotated with spring boot application. Within the main, we pass in arguments if needed, and then spring application is going to run that class with the arguments. A very simple test is provided by the spring initializer. It is annotated with spring boot test. This indicates to spring to bring up a test context. All this test does is make sure that spring starts without errors. Now the package structure of your classes. Spring package structure. There's a recommendation in the spring documentation. It's really not very detailed. There's no official standard, but you should be organizing your application classes into packages. That's going to make it easy for yourself and other people on your team to navigate. A common pattern is to organize by type and function. Here you can see on the left hand side of the screen. This is from a project that I've been working on. This is a project for the

company that I'm working on, and you can see that I'm organized into Authserver data intake and payment processor. And these are all major application modules within Authserver. I have a package for bootstrap that's initialization classes, a config. This is where I put spring configuration classes controller for web controllers domain. As for my ORM objects, I have a special package called keys. Mappers are where I'm going to be placing type converters from. Map struct model is for dtos working with the web layer. And then we also have repositories for spring data, JPA repositories, and then also a services package. This can vary a little bit from project to project, but this is a pretty common pattern to have things organized by function into different packages. When you are working on the project, you can easily locate different components. When we initialize a project with spring initializer, we will also have Maven wrapper. This is a utility that is going to allow you to run Maven from the command line, without having installed locally. In the project directory, you'll have a file called Maven or MVM. This is a shell script that you can utilize with macOS or Linux, or Mvn or cmd. This is a shell script that executes in windows environments. Also, you will see a hidden directory called MVM. This is a directory for Maven wrapper properties. So for example, if you did not have Maven installed on your system, you can still run Maven commands using the command Maven MVM w build. This would execute the build of Maven project.

Maven Standard Directory Layout

- Maven Standard Directory Layout - Established by Maven with the 2.0 release in 2005
- Industry Standard for Java Projects
- Also used by other Java build tools, such as Gradle
- Project Directory:
  - pom.xml - Maven Build File with project metadata, dependencies, build plugins, etc.
  - README.md - Markdown file with project information, also shown on root page of GitHub
  - /target - Project build artifacts
  - /src - All source files for building the project
    - /main - Application Source Files
      - /java - Java Source Files

        - /resources - Non Java source files, such as property files or images
    - /test - Application Source Files
        - /java - Java Source Files for Unit Tests
        - /resources - Non Java source files, such as property files or images for Unit Tests

```
project/
 ├── .gitignore
 ├── pom.xml                      #
 ├── /src
 |    ├── /main
 |    |    ├── /java
 |    |    └── /resources
 |    └── /test
 |         ├── /java
 |         └── /resources
 └── README.md
```

Spring Boot Specific Files

- /src/main/resources/application.properties - Key Value properties file
- /src/main/resources/application.yaml- YAML formatted properties file
- /src/test/resources/application.properties - Key Value properties file for Unit Tests
- /src/tes/resources/application.yaml- YAML formatted properties file for Unit Tests
- /src/main/resources/static - Static Web files, served from root url (images, javascript, etc.)
- /src/main/resources/templates - Web Application Template files
- /src/main/resources/db/migration - Database Migration Scripts

Spring Boot Default Classes

- In the default package of your project (com.myproject.example) is the Spring Boot Main Class

    - The Main Class is annotated with `@SpringBootApplication`
    - The main method is executed at run time to start the Spring Boot Application
- Important - Spring Boot will scan for components in the default package and all child packages
    - Components not under the default package will NOT be picked up
- Projects created using the Spring Initializer will have a default test class for the application
    - Simple test to verify Spring is able to start without errors.

Spring Boot Main Class Example

```
@SpringBootApplication
public class DemoApplication {

  public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
  }

}
```

Spring Boot Test Example

```
@SpringBootTest
class DemoApplicationTests {

  @Test
  void contextLoads() {
  }

}
```

Package Structure

- Spring's package structure recommendation is somewhat vague, and not detailed.

- There is no official standard for the Spring Boot Package Structure
- You should organize the application classes into packages which make it easy for yourself and others to navigate
- A common pattern is to organize by type/function

Maven Wrapper

- Maven Wrapper is a utility allowing you to run Maven
- without having it installed locally
- Project Directory:
  - mvnw - Shell script for macOS/Linux
  - mvnw.cmd - Shell script for Windows
  - /.mvn - Directory for Maven Wrapper properties
- Example - `./mvnw build` - Runs Maven Build of project

Anatomia de um Projeto Spring Boot

Na próxima aula, iremos criar um projeto Spring Boot. Mas, antes disso, vamos analisar a anatomia de um projeto Spring Boot.

Por padrão, o Spring Boot segue o **layout de diretórios padronizado do Maven**. Esse padrão foi estabelecido pelo Maven com o lançamento da versão 2.0, em 2005, e acabou se tornando um padrão da indústria para projetos Java. Antes disso, não havia um padrão claro; cada projeto era estruturado de forma diferente. Com o tempo, esse modelo se consolidou como referência para projetos Java e também passou a ser utilizado por outras ferramentas de build, como o Gradle.

No diretório raiz do projeto, você terá um arquivo **pom.xml**. Esse é o principal arquivo de build do Maven, contendo os metadados do projeto, dependências, plugins de build e diversas outras configurações que podem ser realizadas com o Maven. Normalmente, também haverá um arquivo **README**, que é um arquivo Markdown onde você pode inserir informações do projeto. Ao versionar o projeto no GitHub, esse arquivo será exibido na página principal do repositório.

Dentro do projeto Maven, existe o diretório **target**, que é onde o Maven colocará todos os artefatos gerados durante o build. Sempre que o projeto for

compilado, os artefatos resultantes serão armazenados nesse diretório.

O diretório **src** contém todos os arquivos-fonte do projeto. Dentro dele, temos:

- **src/main**, que contém os arquivos-fonte da aplicação:
  - **src/main/java**, com os arquivos-fonte Java.
  - **src/main/resources**, com arquivos que não são Java, como arquivos de propriedades, imagens, entre outros.
- **src/test**, que contém os arquivos-fonte de testes:
  - **src/test/java**, com os arquivos Java de testes unitários.
  - **src/test/resources**, com arquivos não Java, como propriedades ou imagens específicas para testes unitários.

Para ajudar a visualizar a estrutura do projeto, temos no diretório principal:

- **pom.xml**, o arquivo de build;
- **README.md**;
- **.gitignore**, criado automaticamente ao gerar o projeto com o Spring Initializr, para ignorar arquivos e diretórios específicos no Git;
- Os diretórios **src** e **target**.

Dentro do diretório **src**, temos **main** e **test**, e em ambos existem os subdiretórios **java** e **resources**.

O Spring Boot adiciona alguns arquivos específicos em **src/main/resources**. Você terá um arquivo **application.properties** ou **application.yaml**. É recomendado utilizar apenas um dos dois, e não ambos. Esses arquivos permitem definir propriedades que serão utilizadas pela aplicação Spring. Da mesma forma, em **src/test/resources**, você pode ter **application.properties** ou **application.yaml** específicos para os testes unitários.

O Spring Boot também define alguns diretórios especiais:

- **src/main/resources/static**: local onde são colocados arquivos web estáticos, como imagens, JavaScript, entre outros. Por padrão, o Spring Boot configura a aplicação para servir esses arquivos a partir da URL raiz.
- **src/main/resources/templates**: local onde são armazenados os arquivos de templates da aplicação web.

- **src/main/resources/db/migration**: local destinado aos scripts de migração de banco de dados.

No pacote padrão do seu projeto — por exemplo, `com.myproject.example` — estará localizada a **classe principal do Spring Boot**. Essa classe é anotada com `@SpringBootApplication`, indicando ao framework que se trata da classe principal que será executada no momento da inicialização. O método `main` é responsável por iniciar a aplicação Spring Boot, acionando a autoconfiguração e o bootstrap da aplicação.

É muito importante destacar que o Spring Boot realiza o **scan de componentes** apenas no pacote padrão e em todos os seus subpacotes. Um erro bastante comum é adicionar componentes Spring fora desse pacote padrão, fazendo com que eles não sejam detectados automaticamente. É possível configurar o Spring para escanear outros pacotes, mas esse não é o comportamento padrão.

Projetos criados com o **Spring Initializr** já incluem uma classe de teste padrão. Trata-se de um teste simples que apenas verifica se o contexto do Spring é carregado corretamente, garantindo que a aplicação inicializa sem erros.

A seguir, temos um exemplo de uma aplicação Spring Boot. Observe a classe principal anotada com `@SpringBootApplication`. No método `main`, os argumentos são repassados para o `SpringApplication.run`, que inicia a aplicação.

Também é fornecido um teste simples pelo Spring Initializr, anotado com `@SpringBootTest`. Essa anotação indica ao Spring que um contexto de teste deve ser inicializado. O teste apenas verifica se a aplicação sobe sem falhas.

Estrutura de Pacotes

A documentação do Spring fornece recomendações sobre estrutura de pacotes, porém essas orientações não são muito detalhadas e não existe um padrão oficial. Ainda assim, é altamente recomendável organizar as classes da aplicação em pacotes de forma clara, facilitando a navegação tanto para você quanto para outros membros da equipe.

Um padrão bastante comum é organizar os pacotes por **tipo ou função**. No exemplo apresentado, o projeto é dividido em grandes módulos da aplicação, como *authserver*, *data intake* e *payment processor*. Dentro desses módulos, há pacotes específicos, como:

- **bootstrap**: classes de inicialização;

- **config**: classes de configuração Spring;
- **controller**: controllers web;
- **domain**: objetos de domínio utilizados pelo ORM;
- **keys**: classes relacionadas a chaves;
- **mappers**: conversores de tipo, por exemplo, utilizando MapStruct;
- **model**: DTOs utilizados na camada web;
- **repositories**: repositórios Spring Data JPA;
- **services**: camada de serviços.

Essa organização pode variar de projeto para projeto, mas esse modelo é bastante comum e facilita a localização dos diferentes componentes da aplicação.

Maven Wrapper

Ao inicializar um projeto com o Spring Initializr, também será incluído o **Maven Wrapper**. Trata-se de um utilitário que permite executar comandos Maven sem que o Maven esteja instalado localmente no sistema.

No diretório do projeto, você encontrará:

- **mvnw**: script shell para macOS e Linux;
- **mvnw.cmd**: script para ambientes Windows;
- **.mvn**: diretório contendo as propriedades do Maven Wrapper.

Por exemplo, mesmo sem o Maven instalado, é possível executar o build do projeto com o comando:

```
./mvnw build
```

Esse comando executará o build completo do projeto Maven utilizando o Maven Wrapper.

Se quiser, posso **transformar essa explicação em um resumo visual**, **mapa mental**, ou **comparar essa estrutura com projetos reais de mercado usando Spring Boot 4**.